

DEPLOYMENT

Securely deploying AI agents

A guide to securing Claude Code and Agent SDK deployments with isolation, credential management, and network controls

Claude Code and the Agent SDK are powerful tools that can execute code, access files, and interact with external services on your behalf. Like any tool with these capabilities, deploying them thoughtfully ensures you get the benefits while maintaining appropriate controls.

Unlike traditional software that follows predetermined code paths, these tools generate their actions dynamically based on context and goals. This flexibility is what makes them useful, but it also means their behavior can be influenced by the content they process: files, webpages, or user input. This is sometimes called prompt injection. For example, if a repository's README contains unusual instructions, Claude Code might incorporate those into its actions in ways the operator didn't anticipate. This guide covers practical ways to reduce this risk.

The good news is that securing an agent deployment doesn't require exotic infrastructure. The same principles that apply to running any semi-trusted code apply here: isolation, least privilege, and defense in depth. Claude Code includes several security features that help with common concerns, and this guide walks through these along with additional hardening options for those who need them.

Not every deployment needs maximum security. A developer running Claude Code on their laptop has different requirements than a company processing customer data in a multi-tenant environment. This guide presents options ranging from Claude Code's built-in security features to hardened production architectures, so you can choose what fits your situation.

Threat model

Agents can take unintended actions due to prompt injection (instructions embedded in content they process) or model error. Claude models are designed to resist this; see the [model overview](#) and the system card for the model you deploy for evaluation details.

Defense in depth is still good practice though. For example, if an agent processes a malicious file that instructs it to send customer data to an external server, network controls can block that request

entirely.

Built-in security features

Claude Code includes several security features that address common concerns. See the [security documentation](#) for full details.

- **Permissions system:** Every tool and bash command can be configured to allow, block, or prompt the user for approval. Use glob patterns to create rules like “allow all npm commands” or “block any command with sudo”. Organizations can set policies that apply across all users. See [permissions](#).
- **Command parsing for permissions:** Before executing bash commands, Claude Code parses them into an AST and matches the result against your permission rules. Commands that cannot be parsed cleanly, or that do not match an allow rule, require explicit approval. A small set of constructs such as `eval` always require approval regardless of allow rules. This is a permission gate, not a sandbox; it does not infer whether a command is dangerous from its target path or effects.
- **Web search summarization:** Search results are summarized rather than passing raw content directly into the context, reducing the risk of prompt injection from malicious web content.
- **Sandbox mode:** Bash commands can run in a sandboxed environment that restricts filesystem and network access. See the [sandboxing documentation](#) for details.

Security principles

For deployments that require additional hardening beyond Claude Code’s defaults, these principles guide the available options.

Security boundaries

A security boundary separates components with different trust levels. For high-security deployments, you can place sensitive resources (like credentials) outside the boundary containing the agent. If something goes wrong in the agent’s environment, resources outside that boundary remain protected.

For example, rather than giving an agent direct access to an API key, you could run a proxy outside the agent’s environment that injects the key into requests. The agent can make API calls, but it never sees the credential itself. This pattern is useful for multi-tenant deployments or when processing

untrusted content.

Least privilege

When needed, you can restrict the agent to only the capabilities required for its specific task:

Resource	Restriction options
Filesystem	Mount only needed directories, prefer read-only
Network	Restrict to specific endpoints via proxy
Credentials	Inject via proxy rather than exposing directly
System capabilities	Drop Linux capabilities in containers

Defense in depth

For high-security environments, layering multiple controls provides additional protection. Options include:

- Container isolation
- Network restrictions
- Filesystem controls
- Request validation at a proxy

The right combination depends on your threat model and operational requirements.

Isolation technologies

Different isolation technologies offer different tradeoffs between security strength, performance, and operational complexity.

 In all of these configurations, Claude Code (or your Agent SDK application) runs inside the isolation boundary (the sandbox, container, or VM). The security controls described below restrict what the agent can access from within that boundary.

Technology	Isolation strength	Performance overhead	Complexity
Sandbox runtime	Good (secure defaults)	Very low	Low

Technology	Isolation strength	Performance overhead	Complexity
Containers (Docker)	Setup dependent	Low	Medium
gVisor	Excellent (with correct setup)	Medium/High	Medium
VMs (Firecracker, QEMU)	Excellent (with correct setup)	High	Medium/High

Sandbox runtime

For lightweight isolation without containers, sandbox-runtime enforces filesystem and network restrictions at the OS level.

The main advantage is simplicity: no Docker configuration, container images, or networking setup required. The proxy and filesystem restrictions are built in. You provide a settings file specifying allowed domains and paths.

How it works:

- **Filesystem:** Uses OS primitives (`bubblewrap` on Linux, `sandbox-exec` on macOS) to restrict read/write access to configured paths
- **Network:** Removes network namespace (Linux) or uses Seatbelt profiles (macOS) to route network traffic through a built-in proxy
- **Configuration:** JSON-based allowlists for domains and filesystem paths

Setup:

```
npm install @anthropic-ai/sandbox-runtime
```

Then create a configuration file specifying allowed paths and domains.

Security considerations:

1. **Same-host kernel:** Unlike VMs, sandboxed processes share the host kernel. A kernel vulnerability could theoretically enable escape. For some threat models this is acceptable, but if you need kernel-level isolation, use gVisor or a separate VM.
2. **No TLS inspection:** The proxy allowlists domains based on the client-supplied hostname and does not terminate or inspect encrypted traffic. Code running inside the sandbox can potentially use domain fronting or similar techniques to reach hosts outside the allowlist. If your threat model requires stronger guarantees, configure a TLS-terminating proxy. See the

sandboxing security limitations for more detail. Separately, if the agent has permissive credentials for an allowed domain, ensure it cannot use that domain to trigger other network requests or to exfiltrate data.

For many single-developer and CI/CD use cases, sandbox-runtime raises the bar significantly with minimal setup. The sections below cover containers and VMs for deployments requiring stronger isolation.

Containers

Containers provide isolation through Linux namespaces. Each container has its own view of the filesystem, process tree, and network stack, while sharing the host kernel.

A security-hardened container configuration might look like this:

```
docker run \
  --cap-drop ALL \
  --security-opt no-new-privileges \
  --security-opt seccomp=/path/to/seccomp-profile.json \
  --read-only \
  --tmpfs /tmp:rw,noexec,nosuid,size=100m \
  --tmpfs /home/agent:rw,noexec,nosuid,size=500m \
  --network none \
  --memory 2g \
  --cpus 2 \
  --pids-limit 100 \
  --user 1000:1000 \
  -v /path/to/code:/workspace:ro \
  -v /var/run/proxy.sock:/var/run/proxy.sock:ro \
  agent-image
```

Here's what each option does:

Option	Purpose
<code>--cap-drop ALL</code>	Removes Linux capabilities like <code>NET_ADMIN</code> and <code>SYS_ADMIN</code> that could enable privilege escalation
<code>--security-opt no-new-privileges</code>	Prevents processes from gaining privileges through setuid binaries
<code>--security-opt seccomp=...</code>	Restricts available syscalls; Docker's default blocks ~44, custom profiles can block more

Option	Purpose
<code>--read-only</code>	Makes the container's root filesystem immutable, preventing the agent from persisting changes
<code>--tmpfs /tmp:...</code>	Provides a writable temporary directory that's cleared when the container stops
<code>--network none</code>	Removes all network interfaces; the agent communicates through the mounted Unix socket below
<code>--memory 2g</code>	Limits memory usage to prevent resource exhaustion
<code>--pids-limit 100</code>	Limits process count to prevent fork bombs
<code>--user 1000:1000</code>	Runs as a non-root user
<code>-v ...:/workspace:ro</code>	Mounts code read-only so the agent can analyze but not modify it. Avoid mounting sensitive host directories like <code>~/.ssh</code> , <code>~/.aws</code> , or <code>~/.config</code>
<code>-v .../proxy.sock:...</code>	Mounts a Unix socket connected to a proxy running outside the container (see below)

Unix socket architecture:

With `--network none` , the container has no network interfaces at all. The only way for the agent to reach the outside world is through the mounted Unix socket, which connects to a proxy running on the host. This proxy can enforce domain allowlists, inject credentials, and log all traffic.

This is the same architecture used by sandbox-runtime. Even if the agent is compromised via prompt injection, it cannot exfiltrate data to arbitrary servers. It can only communicate through the proxy, which controls what domains are reachable. For more details, see the Claude Code sandboxing blog post.

Additional hardening options:

Option	Purpose
<code>--usersns-remap</code>	Maps container root to unprivileged host user; requires daemon configuration but limits damage from container escape
<code>--ipc private</code>	Isolates inter-process communication to prevent cross-container attacks

gVisor

Standard containers share the host kernel: when code inside a container makes a system call, it goes directly to the same kernel that runs the host. This means a kernel vulnerability could allow container

escape. gVisor addresses this by intercepting system calls in userspace before they reach the host kernel, implementing its own compatibility layer that handles most syscalls without involving the real kernel.

If an agent runs malicious code (perhaps due to prompt injection), that code runs in the container and could attempt kernel exploits. With gVisor, the attack surface is much smaller: the malicious code would need to exploit gVisor’s userspace implementation first and would have limited access to the real kernel.

To use gVisor with Docker, install the `runsc` runtime and configure the daemon:

```
// /etc/docker/daemon.json
{
  "runtimes": {
    "runsc": {
      "path": "/usr/local/bin/runsc"
    }
  }
}
```

Then run containers with:

```
docker run --runtime=runsc agent-image
```

Performance considerations:

Workload	Overhead
CPU-bound computation	~0% (no syscall interception)
Simple syscalls	~2× slower
File I/O intensive	Up to 10-200× slower for heavy open/close patterns

For multi-tenant environments or when processing untrusted content, the additional isolation is often worth the overhead.

Virtual machines

VMs provide hardware-level isolation through CPU virtualization extensions. Each VM runs its own

kernel, creating a strong boundary. A vulnerability in the guest kernel doesn't directly compromise the host. However, VMs aren't automatically "more secure" than alternatives like gVisor. VM security depends heavily on the hypervisor and device emulation code.

Firecracker is designed for lightweight microVM isolation. It can boot VMs in under 125ms with less than 5 MiB memory overhead, stripping away unnecessary device emulation to reduce attack surface.

With this approach, the agent VM has no external network interface. Instead, it communicates through `vsock` (virtual sockets). All traffic routes through vsock to a proxy on the host, which enforces allowlists and injects credentials before forwarding requests.

Cloud deployments

For cloud deployments, you can combine any of the above isolation technologies with cloud-native network controls:

1. Run agent containers in a private subnet with no internet gateway
2. Configure cloud firewall rules (AWS Security Groups, GCP VPC firewall) to block all egress except to your proxy
3. Run a proxy (such as **Envoy** with its `credential_injector` filter) that validates requests, enforces domain allowlists, injects credentials, and forwards to external APIs
4. Assign minimal IAM permissions to the agent's service account, routing sensitive access through the proxy where possible
5. Log all traffic at the proxy for audit purposes

Credential management

Agents often need credentials to call APIs, access repositories, or interact with cloud services. The challenge is providing this access without exposing the credentials themselves.

The proxy pattern

The recommended approach is to run a proxy outside the agent's security boundary that injects credentials into outgoing requests. The agent sends requests without credentials, the proxy adds them, and forwards the request to its destination.

This pattern has several benefits:

1. The agent never sees the actual credentials
2. The proxy can enforce an allowlist of permitted endpoints
3. The proxy can log all requests for auditing
4. Credentials are stored in one secure location rather than distributed to each agent

Configuring Claude Code to use a proxy

Claude Code supports two methods for routing sampling requests through a proxy:

Option 1: ANTHROPIC_BASE_URL (simple but only for sampling API requests)

```
export ANTHROPIC_BASE_URL="http://localhost:8080"
```

This tells Claude Code and the Agent SDK to send sampling requests to your proxy instead of the Claude API directly. Your proxy receives plaintext HTTP requests, can inspect and modify them (including injecting credentials), then forwards to the real API.

Option 2: HTTP_PROXY / HTTPS_PROXY (system-wide)

```
export HTTP_PROXY="http://localhost:8080"
export HTTPS_PROXY="http://localhost:8080"
```

Claude Code and the Agent SDK respect these standard environment variables, routing all HTTP traffic through the proxy. For HTTPS, the proxy creates an encrypted CONNECT tunnel: it cannot see or modify request contents without TLS interception.

Implementing a proxy

You can build your own proxy or use an existing one:

- **Envoy Proxy**: production-grade proxy with `credential_injector` filter for adding auth headers
- **mitmproxy**: TLS-terminating proxy for inspecting and modifying HTTPS traffic
- **Squid**: caching proxy with access control lists
- **LiteLLM**: LLM gateway with credential injection and rate limiting

Credentials for other services

Beyond sampling from the Claude API, agents often need authenticated access to other services, such as git repositories, databases, and internal APIs. There are two main approaches:

Custom tools

Provide access through an MCP server or custom tool that routes requests to a service running outside the agent's security boundary. The agent calls the tool, but the actual authenticated request happens outside. The tool calls to a proxy which injects the credentials.

For example, a git MCP server could accept commands from the agent but forward them to a git proxy running on the host, which adds authentication before contacting the remote repository. The agent never sees the credentials.

Advantages:

- **No TLS interception:** The external service makes authenticated requests directly
- **Credentials stay outside:** The agent only sees the tool interface, not the underlying credentials

Traffic forwarding

For Claude API calls, `ANTHROPIC_BASE_URL` lets you route requests to a proxy that can inspect and modify them in plaintext. But for other HTTPS services (GitHub, npm registries, internal APIs), the traffic is often encrypted end-to-end. Even if you route it through a proxy via `HTTP_PROXY`, the proxy only sees an opaque TLS tunnel and can't inject credentials.

To modify HTTPS traffic to arbitrary services, without using a custom tool, you need a TLS-terminating proxy that decrypts traffic, inspects or modifies it, then re-encrypts it before forwarding. This requires:

1. Running the proxy outside the agent's container
2. Installing the proxy's CA certificate in the agent's trust store (so the agent trusts the proxy's certificates)
3. Configuring `HTTP_PROXY` / `HTTPS_PROXY` to route traffic through the proxy

This approach handles any HTTP-based service without writing custom tools, but adds complexity around certificate management.

Note that not all programs respect `HTTP_PROXY` / `HTTPS_PROXY`. Most tools (curl, pip, npm, git) do, but some may bypass these variables and connect directly. For example, Node.js `fetch()` ignores

these variables by default; in Node 24+ you can set `NODE_USE_ENV_PROXY=1` to enable support. For comprehensive coverage, you can use [proxychains](#) to intercept network calls, or configure iptables to redirect outbound traffic to a transparent proxy.

❗ A **transparent proxy** intercepts traffic at the network level, so the client doesn't need to be configured to use it. Regular proxies require clients to explicitly connect and speak HTTP CONNECT or SOCKS. Transparent proxies (like Squid or mitmproxy in transparent mode) can handle raw redirected TCP connections.

Both approaches still require the TLS-terminating proxy and trusted CA certificate. They just ensure traffic actually reaches the proxy.

Filesystem configuration

Filesystem controls determine what files the agent can read and write.

Read-only code mounting

When the agent needs to analyze code but not modify it, mount the directory read-only:

```
docker run -v /path/to/code:/workspace:ro agent-image
```

⚠️ Even read-only access to a code directory can expose credentials. Common files to exclude or sanitize before mounting:

File	Risk
<code>.env</code> , <code>.env.local</code>	API keys, database passwords, secrets
<code>~/.git-credentials</code>	Git passwords/tokens in plaintext
<code>~/.aws/credentials</code>	AWS access keys
<code>~/.config/gcloud/application_default_credentials.json</code>	Google Cloud ADC tokens
<code>~/.azure/</code>	Azure CLI credentials
<code>~/.docker/config.json</code>	Docker registry auth tokens
<code>~/.kube/config</code>	Kubernetes cluster credentials
<code>.npmrc</code> , <code>.pypirc</code>	Package registry tokens
<code>*-service-account.json</code>	GCP service account keys
<code>*.pem</code> , <code>*.key</code>	Private keys

Consider copying only the source files needed, or using `.dockerignore` -style filtering.

Writable locations

If the agent needs to write files, you have a few options depending on whether you want changes to persist:

For ephemeral workspaces in containers, use `tmpfs` mounts that exist only in memory and are cleared when the container stops:

```
docker run \  
  --read-only \  
  --tmpfs /tmp:rw,noexec,nosuid,size=100m \  
  --tmpfs /workspace:rw,noexec,size=500m \  
  agent-image
```

If you want to review changes before persisting them, an overlay filesystem lets the agent write without modifying underlying files. Changes are stored in a separate layer you can inspect, apply, or discard. For fully persistent output, mount a dedicated volume but keep it separate from sensitive

directories.

Further reading

- [Claude Code security documentation](#)
- [Hosting the Agent SDK](#)
- [Handling permissions](#)
- [Sandbox runtime](#)
- [The Lethal Trifecta for AI Agents](#)
- [OWASP Top 10 for LLM Applications](#)
- [Docker Security Best Practices](#)
- [gVisor Documentation](#)
- [Firecracker Documentation](#)